

```
<!--SESIÓN TEÓRICO-PRÁCTICA-->
```

Protocolo HTTP y Curl {

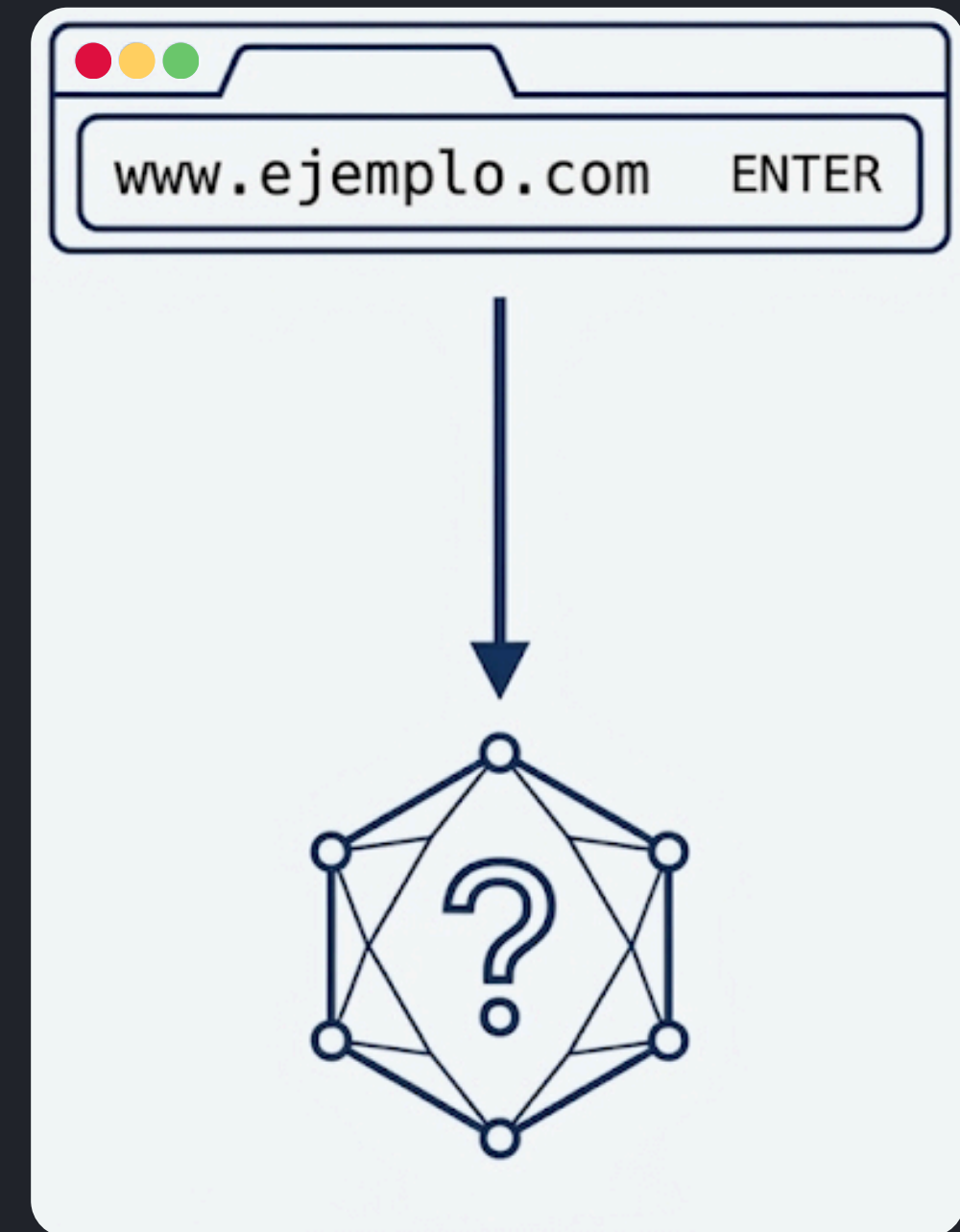
```
<Tema="Protocolos, Métodos y Códigos"/>
```

}



¿Qué ocurre al presionar Enter? {

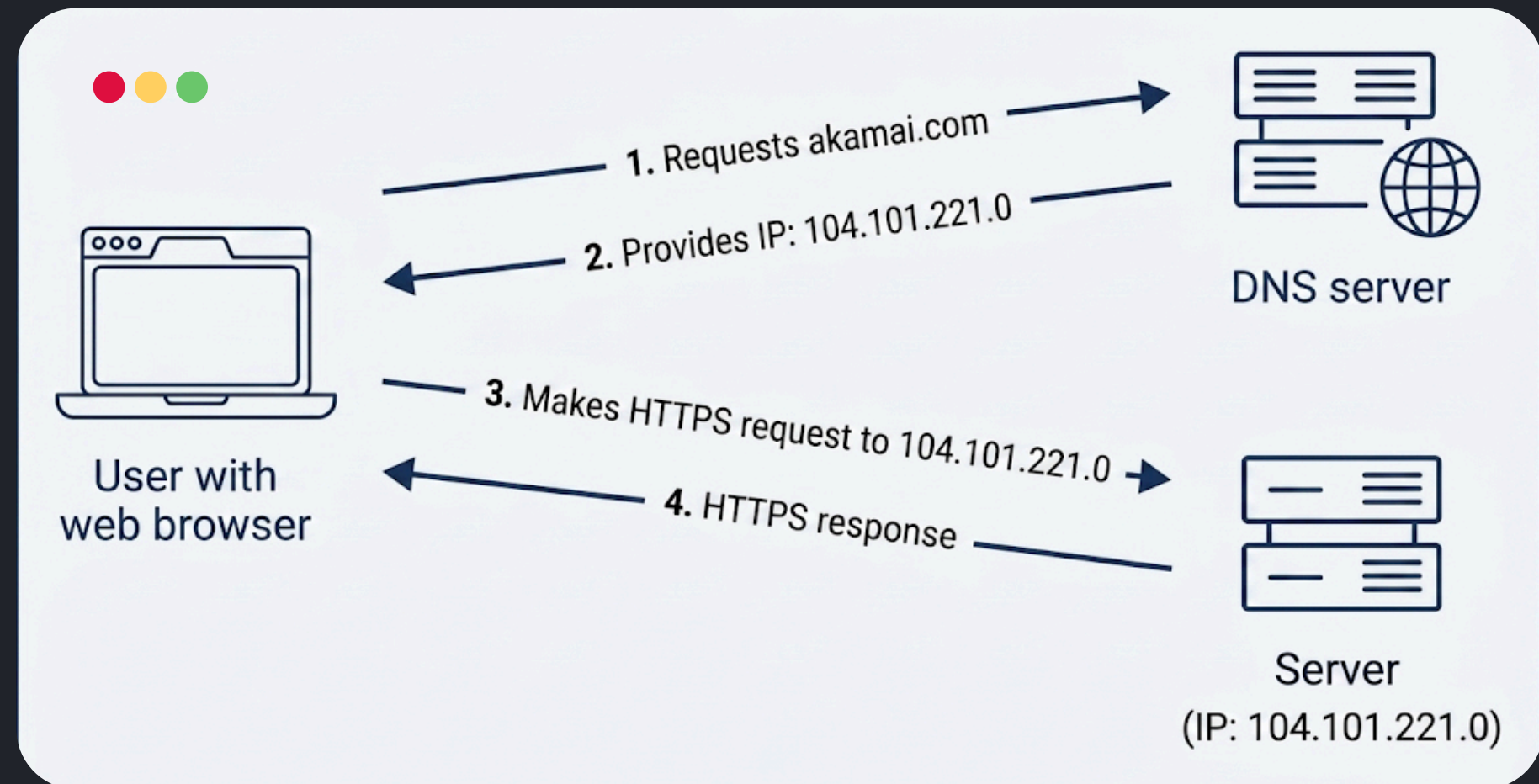
- Parece magia, pero en realidad es una secuencia precisa de operaciones de red.
- La Web no es abstracta: depende de infraestructura física, servidores, enrutadores, cables submarinos.
- Cada clic traduce una intención humana en instrucciones que las máquinas pueden ejecutar.
- Y todo comienza con algo tan simple como una cadena de texto: una URL.



}

DNS y el camino al servidor {

- DNS (Domain Name System): el directorio telefónico de internet.
- Traduce dominios legibles en direcciones IP (IPv4 / IPv6).
- La IP identifica de forma única a cada servidor en la red global.
- Con la IP resuelta, se abre un túnel TCP: una conexión estable hacia el servidor.



}

Desmenuzando el localizador (URL) {

https : // **www.api.com** : **443** / **users/list** ? **sort=asc** # **top**

Esquema

Protocolo de comunicación

Host

Quién atiende la solicitud

Puerto

Puerta de entrada
(80 / 443)

Path

Recurso solicitado

Query

Parámetros adicionales

Fragment

Posición dentro del recurso

}

¿Qué es HTTP? {

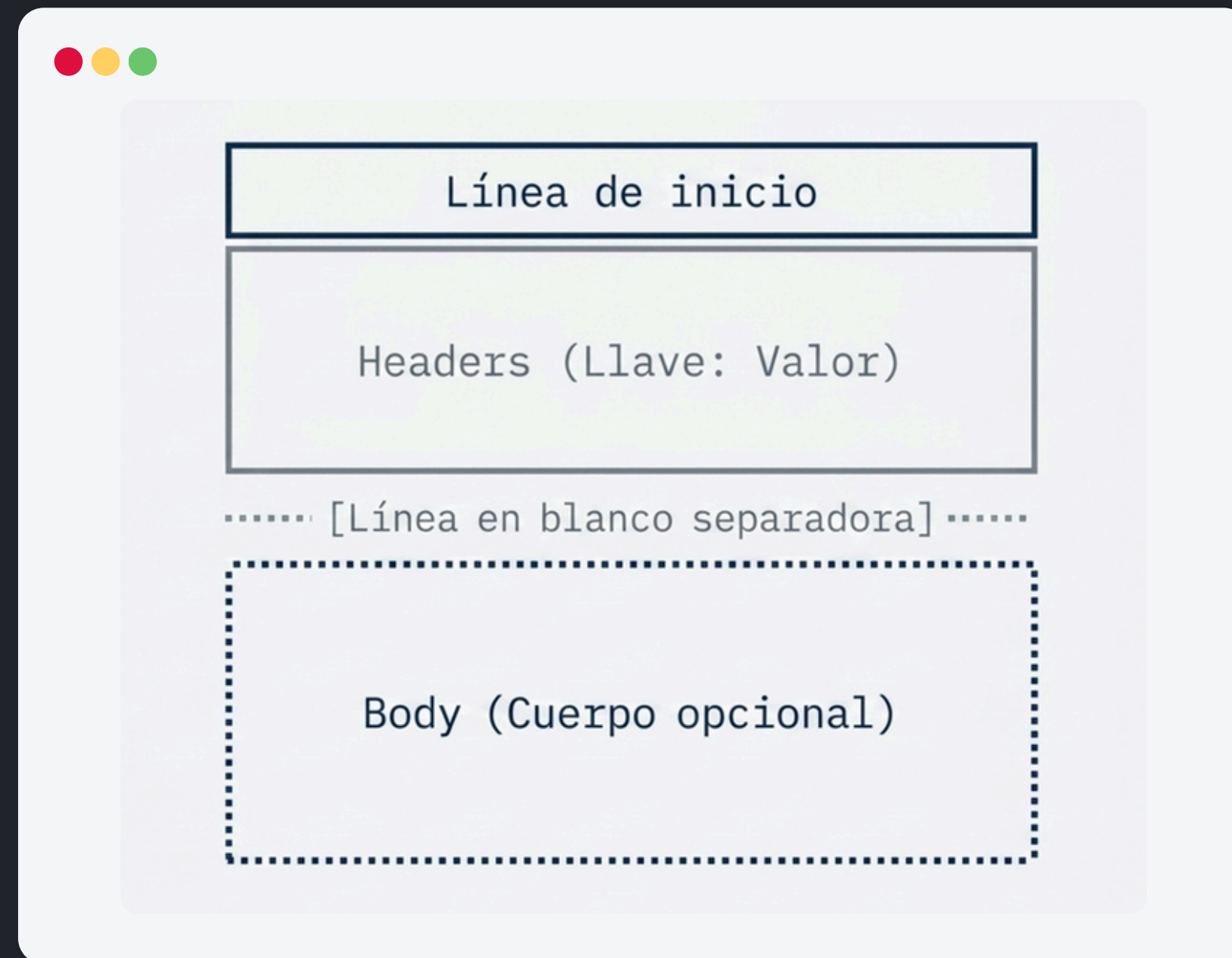
HyperText Transfer

Protocol: el protocolo de aplicación que hace funcionar la Web. Define cómo un cliente solicita un recurso y cómo un servidor responde a esa solicitud.

Toda experiencia de navegar una página, abrir una app o consumir una API descansa, en el fondo, sobre este mismo intercambio de mensajes.

}

¿Qué es HTTP? {



}

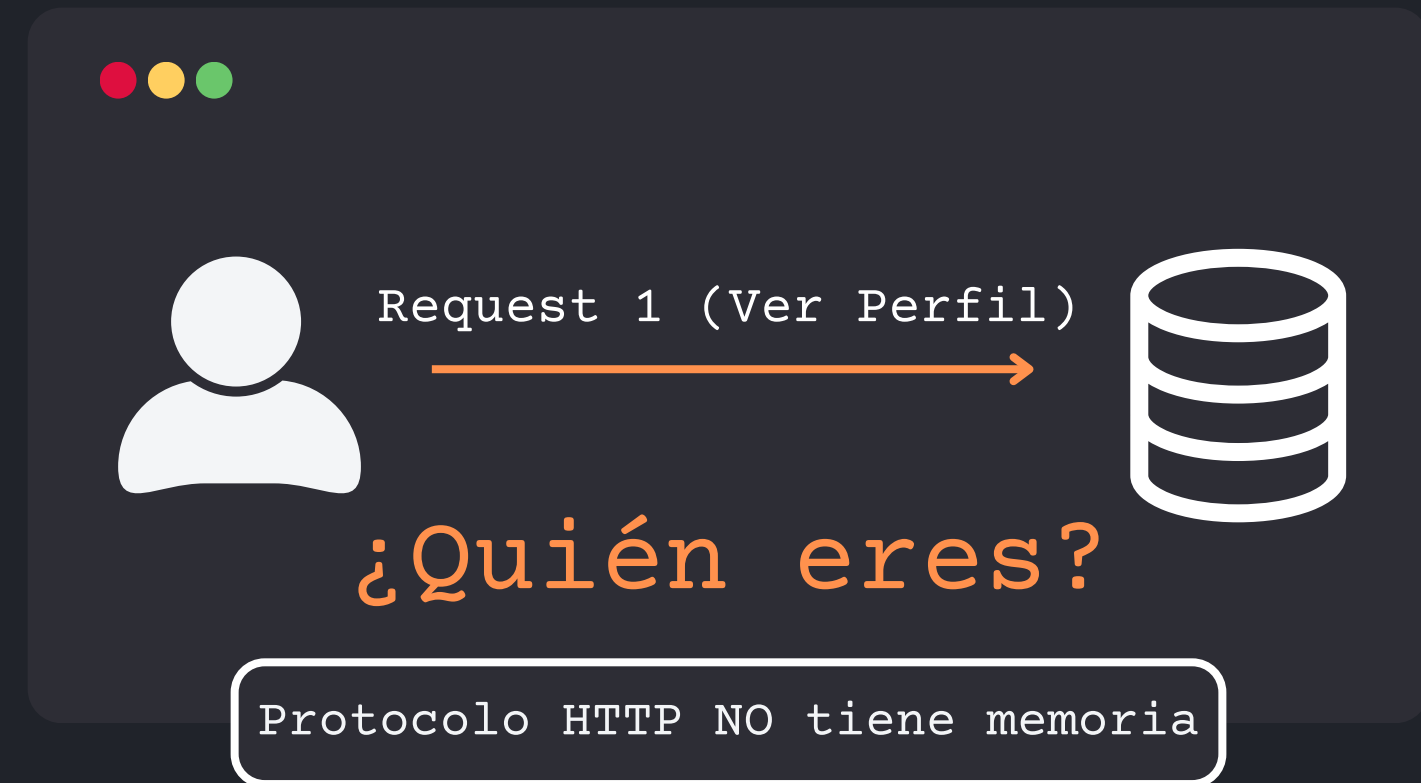
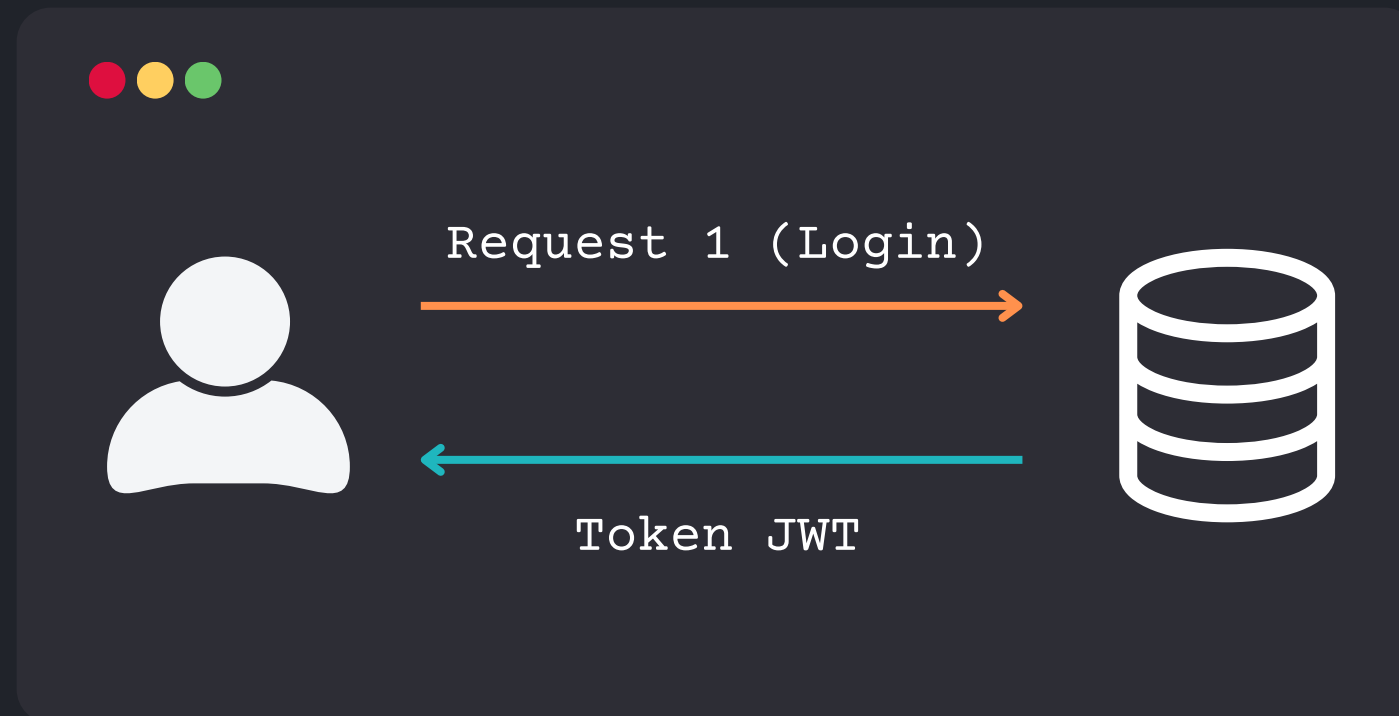
¿Qué es HTTP? {

- Modelo cliente-servidor.
- Intercambio de mensajes: request y response.
- **STATELESS**: cada request es independiente.
- Corre sobre TCP (HTTP/1.1 y 2) o UDP (HTTP/3)



}

¿Qué es Stateless? {



La base de datos sí guarda la información, pero el protocolo **NO**.

}

Versiones de HTTP {

HTTP/1.1

- Mensajes en texto plano.
- Una request a la vez por conexión (bloqueo de cabecera).
- Conexiones keep-alive

Ideal para sistemas simples o heredados (legacy).

HTTP/2

- Formato binario, no texto
- Multiplexación: varias requests en una sola conexión
- Compresión de headers (HPACK) y priorización de streams

Ideal para la web moderna con muchos recursos estáticos.

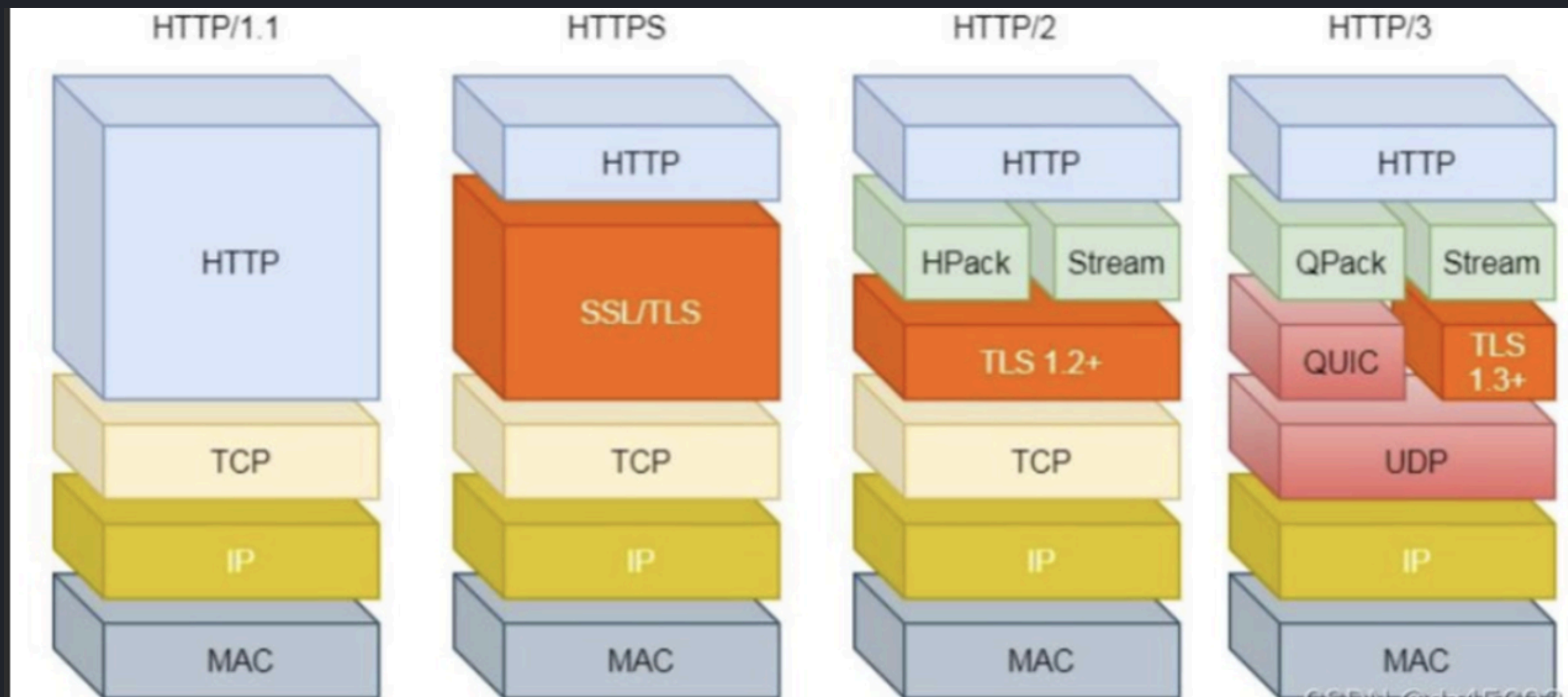
HTTP/3

- Corre sobre QUIC (UDP), no sobre TCP
- Sin bloqueo de cabecera a nivel de transporte
- Menor latencia inicial y más resiliente ante pérdida de paquetes

Ideal para streaming y redes móviles inestables.

}

Versiones de HTTP {



}

TCP, UDP y TLS {

TCP

CONFIABLE

- Entrega los paquetes en secuencia y detecta errores
- Usa acuses de recibo (ACK) para confirmar cada envío
- Establece y mantiene una conexión estable

UDP

VELOZ

- Más rápido que TCP: sin negociación previa
- No garantiza entrega ni orden de los paquetes
- Sin control de errores ni congestión

TLS

SEGURO

- Cifra la comunicación entre cliente y servidor
- Garantiza confidencialidad e integridad de los datos
- HTTP + TLS = HTTPS

}

Métodos HTTP {

MÉTODO	FUNCIÓN	CRUD
GET	Obtiene un recurso sin modificarlo	Read
POST	Crea un nuevo recurso (Formularios)	Create
PUT	Reemplaza un recurso completo	Update
PATCH	Actualiza un recurso parcialmente	Update
DELETE	Elimina un recurso específico	Delete
HEAD	Igual que GET, pero sin cuerpo en la respuesta	
OPTIONS	Consulta qué métodos admite el servidor	-

HTTP Request {



POST /users HTTP/1.1

Host: api.ejemplo.com

Content-Type:

application/json

{"nombre": "Ada"}

Línea de request

Método + ruta + versión del protocolo HTTP.

Headers

Metadatos: tipo de contenido, autenticación, cookies...

Línea vacía

Separa los headers del cuerpo del mensaje.

Body

Los datos enviados al servidor (opcional).

}

HTTP Response {



HTTP/1.1 201 Created

Content-Type:

application/json

Content-Length: 42

{"id": 101, "nombre": "Ada"}

Línea de estado

Versión del protocolo + código + mensaje.

Headers

Metadatos que describen la respuesta.

Línea vacía

Separa los headers del cuerpo del mensaje.

Body

El recurso solicitado, ya creado o actualizado.

}

Códigos de Estado {

2XX

Éxito

200 OK

La solicitud se procesó correctamente.

201 Created

Se creó un nuevo recurso.

3XX

Redirección

301 Moved Permanently

El recurso cambió de URL para siempre.

302 Found

Redirección temporal.

4XX

Error del cliente

400 Bad Request

La solicitud tiene un error de sintaxis.

401 Unauthorized

Falta autenticación válida.

404 Not Found

El recurso no existe.

5XX

Error del servidor

500 Internal Server Error

Falla inesperada en el servidor.

503 Service Unavailable

El servidor no puede atender la solicitud.

}

Curl {



```
$ curl -v https://api.ejemplo.com
```

```
> GET / HTTP/1.1
```

```
> Host: api.ejemplo.com
```

```
> Accept: */*
```

```
>
```

```
< HTTP/1.1 200 OK
```

```
< Content-Type: application/json
```

curl -v

Expone el intercambio de texto puro, sin intermediarios.

Líneas con >

Representan nuestra solicitud: el request del cliente.

Líneas con <

Representan la respuesta del servidor.

Línea en blanco

La cruda anatomía de HTTP: headers, cuerpo y el separador, sin GUI de por medio.

}

```
<!--SESIÓN TEÓRICO-PRÁCTICA-->
```

Actividad {

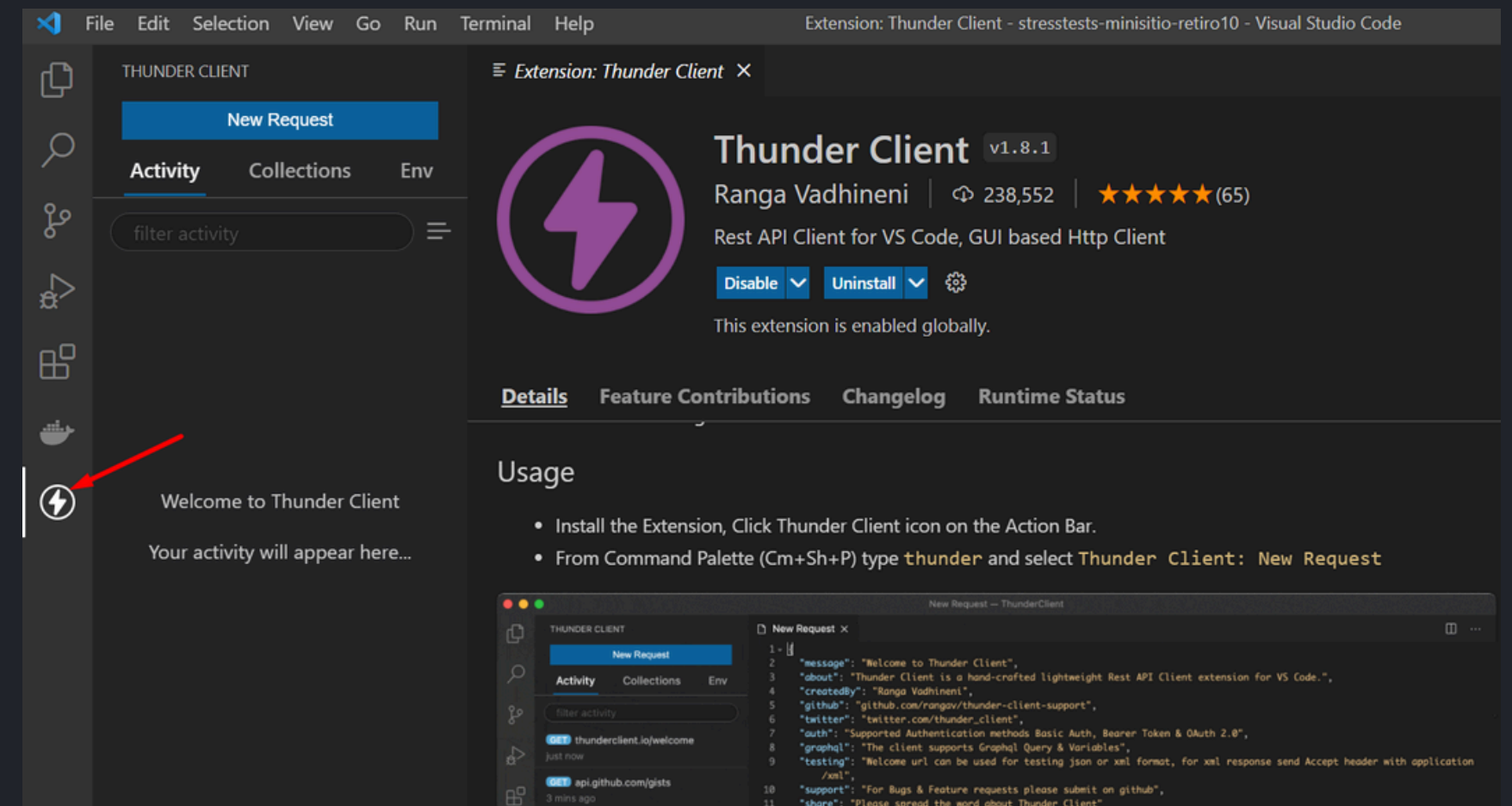
```
<Tema="Thunder Client"/>
```

}



Thunder Client {

- Las interfaces gráficas (Thunder Client, Postman) agilizan el desarrollo diario.
- Separan visualmente los componentes clave: parámetros, headers y body.
- Permiten guardar colecciones de peticiones para probar flujos completos.
- Dan control total sobre la comunicación, sin programar un frontend.



}

Actividad {

- Objetivo: aplicar los conceptos de HTTP consumiendo una API externa real.
- Herramienta: Visual Studio Code + extensión Thunder Client (o Postman).
- API mock: JSONPlaceholder — `jsonplaceholder.typicode.com`
- Formato: trabajaremos exclusivamente leyendo y enviando objetos JSON.

```
Content-Type: application/json
{
  "userId": 1,
  "id": 1,
  "title": "Aprender HTTP de verdad",
  "completed": false
}
```

}

Actividad {

Punto 1 - GET

Recuperar exclusivamente los posts del usuario 2.

Pista: utilicen un query param en la URL (?userId=2).

GET /posts?userId=2

Punto 2 - POST

Crear un nuevo recurso en el servidor.

Pista: método POST apuntando a /posts, con un JSON bien formateado en el body.

POST /posts

}

Actividad {

Punto 3 - HEADER

Interceptar y negociar el formato de respuesta.

Acción: en Thunder Client, modifiquen manualmente el header Accept.

Accept: text/html

Punto 4 - Exploración

Comprobar la respuesta del servidor ante solicitudes inválidas.

Acción: provoquen intencionalmente un error 404 Not Found en la API.

404 Not Found

}

Entrega Final {

¿Cómo es técnicamente posible que una aplicación mantenga la sesión iniciada al navegar entre distintas páginas... si HTTP es estrictamente "stateless" y el servidor olvida cada petición?

ENTREGABLE

Suban una captura de pantalla de cada uno de los puntos completados al Canvas.

}

```
<!--Estudio Shonos-->
```

Gracias {

```
<Por="Alba Castro"/>
```

}